Bereich: Probe-PE

SpeedyQuiz	**Musterlösung**
Package: `de.dhbwka.java.exams. speedyquiz`	**SpeedyQuiz**

```
package de.dhbwka.java.exams.speedyquiz;

import java.io.BufferedReader;
import java.io.FileReader;
import java.util.ArrayList;
import java.util.List;

import javax.swing.JOptionPane;
import javax.swing.UIManager;

/**
 * Speedy Quiz
 */
public class SpeedyQuiz {

	/**
	 * Main method, entry point of application
	 *
	 * @param args CLI arguments
	 */
	public static void main(String[] args) {
		// Set cross platform LAF to get colors to work for sure on MacOS
		try {

	UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
		} catch (Exception e) {
		}

		List<Question> questionPool = loadQuestions();

		try {
			Game game = new Game(questionPool, 10);

			game.registerClient( new GameTerm("Ben", game) );
			game.registerClient( new GameTerm("Mia", game) );

			game.startGame();
		} catch (GameException e) {
			JOptionPane.showMessageDialog(null, e.getMessage(), "Game Error",
JOptionPane.ERROR_MESSAGE);
		}

	}

	/**
	 * Load questions
	 *
	 * @return questions to load
	 */
	public static List<Question> loadQuestions(){
		List<Question> questions = new ArrayList<>();

		// replace this code
		// Generates 15 dummy questions, the first answer is always correct
//		for (int i = 1; i <= 15; i++) {
//			String line = String.format("Generated question %s;Answer 1;Answer
```

```
2;Answer 3;Answer 4;0", i);
//                    Question q = parseQuestion(line);
//                    if ( q != null ){
//                           questions.add(q);
//                    }
//             }
               try ( BufferedReader br = new BufferedReader(new
FileReader("questions.txt"))) {
                    while ( br.ready() ){
                           Question q = parseQuestion(br.readLine());
                           if ( q != null ){
                                  questions.add(q);
                           }
                    }
               } catch (Exception e) {
                    JOptionPane.showMessageDialog(null, e.getMessage(), "Error loading
questions", JOptionPane.ERROR_MESSAGE);
               }

               return questions;
        }

        /**
         * Parse a question from input line
         *
         * @param line line to parse
         * @return created question instance
         */
        public static Question parseQuestion( String line ){
               try {
                    String[] parts = line.split(";");
                           if ( parts.length == 6 ){
                           String[] answers = new String[4];
                           System.arraycopy(parts, 1, answers, 0, 4);
                           return new Question(parts[0], answers,
Integer.parseInt(parts[5]));
                    }
               } catch (Exception e) {
               }
               return null;
        }

}

// classes afterwards in alphabetical order
```

```
package de.dhbwka.java.exams.speedyquiz;

import java.io.FileWriter;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedList;
import java.util.List;

import javax.swing.JOptionPane;

/**
 * Game management
 */
public class Game {

        /**
```

```
         * Current question index, initialized with <code>-1</code>
         * so increment at beginning of {@link #nextQuestion()}
         * will result in index 0
         */
        private int currentQuestionIndex = -1;

        /**
         * List of questions
         */
        private List<Question> questions = new LinkedList<>();

        /**
         * Client list
         */
        private List<GameClient> clients = new ArrayList<>();

        /**
         * Time when {@link #startGame()} is called
         */
        private long startTime;

        /**
         * Create game
         *
         * @param questionPool question pool
         * @param count count of questions to use
         * @throws GameException if too few questions are available
         */
        public Game(List<Question> questionPool, int count) throws GameException {
                if (questionPool.size() < count) {
                        throw new GameException("Too few questions available!");
                }

                Collections.shuffle(questionPool);
                for (int i = 0; i < count; i++) {
                        questions.add(questionPool.get(i));
                }
        }

        /**
         * Register a client
         *
         * @param client client to register
         */
        public void registerClient( GameClient client ){
                if ( this.currentQuestionIndex == -1 ){ // -1 only if game not started
                        this.clients.add(client);
                }
        }

        /**
         * Next question
         */
        private void nextQuestion(){
                this.currentQuestionIndex++;
                if ( this.currentQuestionIndex < this.getQuestionsCount() ){
                        for ( GameClient t : this.clients ){
                                t.setQuestion( this.currentQuestionIndex,
this.getCurrentQuestion() );
                        }
                        this.startQuestionCountdown();
                } else {
                        for ( GameClient client : this.clients ){
                                client.gameIsOver();
```

```
                        }
                        this.doScoreAndLog();
                }
        }

        /**
         * Do score and logging
         */
        private void doScoreAndLog() {
                long durationSec = (System.currentTimeMillis() - this.startTime) / 1000;

                List<String> scores = new LinkedList<>();
                for ( GameClient t : this.clients ){
                        scores.add(t.getPlayerName() + " (" + t.getPoints() + ")");
                }
                String score = "Game finished after " + durationSec + " seconds, score: " +
String.join(", ", scores);

                try ( PrintWriter pw = new PrintWriter(new FileWriter("highscore.txt",
true)) ) {
                        pw.println(score);
                } catch (Exception e) {
                        e.printStackTrace();
                }

                JOptionPane.showMessageDialog(null, score);
        }

        /**
         * Get current question
         *
         * @return current question
         */
        private Question getCurrentQuestion(){
                return this.questions.get(this.currentQuestionIndex);
        }

        /**
         * Get count of questions
         *
         * @return count of questions
         */
        public int getQuestionsCount(){
                return this.questions.size();
        }

        /**
         * Answer selected
         *
         * @param client client
         * @param index index of answer (0-based)
         */
        public void answerSelected( GameClient client, int index ){
                boolean correct = this.getCurrentQuestion().getCorrectIndex() == index;

                for ( GameClient t : this.clients ){
                        if ( t == client ){
                                t.setAnswerState(this.currentQuestionIndex, correct ?
Status.CORRECT : Status.WRONG );
                        } else {
                                t.setAnswerState(this.currentQuestionIndex, Status.NO_ANSWER
);
                        }
                }
```

```
                this.nextQuestion();
        }

        /**
         * Start the game
         */
        public void startGame(){
                this.startTime = System.currentTimeMillis();
                this.nextQuestion();
        }

        /**
         * Start question countdown
         */
        private void startQuestionCountdown(){
                final int questionIndexAtStart = this.currentQuestionIndex;
                new Thread(new Runnable() {

                        @Override
                        public void run() {
                                int remaining = 10;
                                while ( questionIndexAtStart == currentQuestionIndex &&
remaining >= 0 ){
                                        if ( remaining == 0 ){
                                                answerSelected(null, -1);
                                        } else {
                                                for ( GameClient t : Game.this.clients ){
                                                        t.setRemainingSeconds(remaining);
                                                }
                                                remaining--;
                                        }
                                        try {
                                                Thread.sleep(1000);
                                        } catch (InterruptedException e) {
                                        }
                                }
                        }
                }, "q-"+questionIndexAtStart+"-countdown").start();
        }

}
```

```
package de.dhbwka.java.exams.speedyquiz;

/**
 * Game client interface
 */
public interface GameClient {

        /**
         * Set the answer state
         *
         * @param questionIndex index of question (0-based)
         * @param status status to set
         */
        public void setAnswerState(int questionIndex, Status status);

        /**
         * Get points
         *
         * @return points
         */
        public int getPoints();
```

```
        /**
         * Set the question
         *
         * @param questionIndex index of question
         * @param q question itself
         */
        public void setQuestion(int questionIndex, Question q);

        /**
         * Set remaining seconds
         *
         * @param seconds seconds to set
         */
        public void setRemainingSeconds(int seconds);

        /**
         * Get the player name
         *
         * @return player name
         */
        public String getPlayerName();

        /**
         * To be called when game is over
         */
        public void gameIsOver();

}
```

```
package de.dhbwka.java.exams.speedyquiz;

/**
 * Game exception
 */
@SuppressWarnings("serial")
public class GameException extends Exception {

        /**
         * Create exception
         *
         * @param message message
         */
        public GameException(String message) {
                super(message);
        }

}
```

```
package de.dhbwka.java.exams.speedyquiz;

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.WindowConstants;

/**
 * Game terminal
```

```
 */
@SuppressWarnings("serial")
public class GameTerm extends JFrame implements GameClient {

	/**
	 * The game reference
	 */
	private final Game game;

	/**
	 * Name of the player
	 */
	private final String playerName;

	/**
	 * The button for the answers
	 */
	private final JButton[] btnAnswers = new JButton[4];

	/**
	 * Label for question text
	 */
	private final JLabel lblQuestionText = new JLabel("", JLabel.CENTER);

	/**
	 * Label for countdown
	 */
	private final JLabel lblCountdown = new JLabel("", JLabel.CENTER);

	/**
	 * Array with all question number labels
	 */
	private final QuestionNumberLabel[] lblQNumbers;

	/**
	 * Create game terminal
	 *
	 * @param playerName name of player
	 * @param game game reference
	 */
	public GameTerm( String playerName, Game game ) {
		super(playerName);
		this.game = game;
		this.playerName = playerName;
		this.lblQNumbers = new QuestionNumberLabel[game.getQuestionsCount()];

		this.initUI();
	}

	/**
	 * Initialize user interface
	 */
	private void initUI() {
		this.setLayout(new BorderLayout(5, 5));

		JPanel panQNumLabels = new JPanel(new GridLayout(1,
this.game.getQuestionsCount(), 5 ,5));
		for (int i = 0; i < this.lblQNumbers.length; i++) {
			this.lblQNumbers[i] = new QuestionNumberLabel(i+1);
			panQNumLabels.add(this.lblQNumbers[i]);
		}

		JPanel panQLabelCountdown = new JPanel(new GridLayout(2, 1));
		panQLabelCountdown.add(this.lblQuestionText);
```

```
		panQLabelCountdown.add(this.lblCountdown);

		JPanel panButtons = new JPanel( new GridLayout(2, 2, 5, 5));
		for (int i = 0; i < this.btnAnswers.length; i++) {
			this.btnAnswers[i] = new JButton("");
			final int index = i;
			this.btnAnswers[i].addActionListener(new ActionListener() {
				@Override
				public void actionPerformed(ActionEvent e) {
					GameTerm.this.game.answerSelected(GameTerm.this,
index);
				}
			});
			panButtons.add( this.btnAnswers[i]);
		}
		this.add( panQNumLabels, BorderLayout.NORTH );
		this.add( panQLabelCountdown, BorderLayout.CENTER );
		this.add( panButtons, BorderLayout.SOUTH );

		this.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
		this.setSize(600, 150);
		this.setVisible(true);
	}

	/**
	 * {@inheritDoc}
	 */
	@Override
	public void setAnswerState( int questionIndex, Status status ){
		this.lblQNumbers[questionIndex].setStatus(status);
	}

	/**
	 * {@inheritDoc}
	 */
	@Override
	public int getPoints(){
		int cnt = 0;
		for ( QuestionNumberLabel l : this.lblQNumbers ) {
			cnt += l.getStatus().getPoints();
		}
		return Math.max(0, cnt);
	}

	/**
	 * {@inheritDoc}
	 */
	@Override
	public void setQuestion( int questionIndex, Question q ){
		this.lblQuestionText.setText(q.getQuestionText());
		this.setAnswerState(questionIndex, Status.ACTIVE);

		String[] answers = q.getAnswers();
		for (int i = 0; i < this.btnAnswers.length; i++) {
			this.btnAnswers[i].setText(answers[i]);
		}
	}

	/**
	 * {@inheritDoc}
	 */
	@Override
	public void setRemainingSeconds( int seconds ){
		this.lblCountdown.setText(Integer.toString(seconds));
```

```
	}

	/**
	 * {@inheritDoc}
	 */
	@Override
	public String getPlayerName() {
		return this.playerName;
	}

	/**
	 * {@inheritDoc}
	 */
	@Override
	public void gameIsOver() {
		for ( JButton btn : this.btnAnswers ){
			btn.setEnabled(false);
		}
	}
}
```

```
package de.dhbwka.java.exams.speedyquiz;

/**
 * Question
 */
public class Question {

	/**
	 * Text of question
	 */
	private final String questionText;

	/**
	 * Answers
	 */
	private final String[] answers;

	/**
	 * Correct index
	 */
	private final int correctIndex;

	/**
	 * Create question
	 *
	 * @param questionText text of question
	 * @param answers answers
	 * @param correctIndex correct index
	 */
	public Question(String questionText, String[] answers, int correctIndex) {
		super();
		this.questionText = questionText;
		this.answers = answers;
		this.correctIndex = correctIndex;
	}

	/**
	 * Get the answers
	 *
	 * @return answers
	 */
	public String[] getAnswers() {
		return answers;
```

```
	}

	/**
	 * Get the correct index
	 *
	 * @return correct index
	 */
	public int getCorrectIndex() {
		return correctIndex;
	}

	/**
	 * Get the question text
	 *
	 * @return question text
	 */
	public String getQuestionText() {
		return questionText;
	}


}
```

```
package de.dhbwka.java.exams.speedyquiz;

import javax.swing.JLabel;

/**
 * Question number label
 */
@SuppressWarnings("serial")
public class QuestionNumberLabel extends JLabel {

	/**
	 * Current status
	 */
	private Status status;

	/**
	 * Create the question number label
	 *
	 * @param number number
	 */
	public QuestionNumberLabel( int number ) {
		super(Integer.toString(number), JLabel.CENTER);
		this.setOpaque(true);
		this.setStatus(Status.PENDING);
	}

	/**
	 * Set the status (and adapt background)
	 *
	 * @param status status to set
	 */
	public void setStatus( Status status ){
		this.status = status;
		this.setBackground( status.getColor() );
	}

	/**
	 * Get the status
	 * @return status
	 */
	public Status getStatus() {
```

```
                return this.status;
        }

}
```

```
package de.dhbwka.java.exams.speedyquiz;

import java.awt.Color;

/**
 * Status
 */
public enum Status {

        /**
         * Active
         */
        ACTIVE(Color.ORANGE,0),

        /**
         * Correct
         */
        CORRECT(Color.GREEN,1),

        /**
         * Wrong
         */
        WRONG(Color.RED,-1),

        /**
         * Pending
         */
        PENDING(Color.WHITE,0),

        /**
         * No answer
         */
        NO_ANSWER(Color.GRAY,0);

        /**
         * Color for status
         */
        private final Color color;

        /**
         * Points for status
         */
        private final int points;

        /**
         * Create status
         *
         * @param color color
         * @param points points
         */
        private Status(Color color, int points) {
                this.color = color;
                this.points = points;
        }

        /**
         * Get color
         *
```

```
         * @return color
         */
        public Color getColor() {
                return color;
        }

        /**
         * Get points
         *
         * @return points
         */
        public int getPoints() {
                return points;
        }
}
```

Mitgelieferte Datei questions.txt

```
Antwort auf die Frage nach dem Leben, dem Universum und dem ganzen Rest;40;41;42;43;2
Der Apfel fällt nicht weit vom...;Zweig;Stamm;Blatt;Stiel;1
Das Runde muss ins...;Runde;Flache;Hohe;Eckige;3
Die Jamaika-Flagge enhält die Farben Schwarz, Gelb und ...;Rot;Grün;Magenta;Blau;1
Der Durchmesser der Erde beträgt...;knapp 13.000km;gut 200.000km;fast 1 Lichtjahr;rund 5.000km;0
Wie ist HTTP-Status-Code für nicht gefundene Seiten?;402;403;404;405;2
Wieviele Drittel ergeben ein Ganzes?;1;2;3;4;2
Berechnen Sie die elektr. Spannung!;P=U*I;E=mc²;U=R*I;F=m*a;2
"Rien ne va plus!" hört man beim...;Black Jack;Skat;Mau-Mau;Roulette;3
1, 1, 2, 3, 4, 8, 13, ...;20;21;22;23;1
DuckTales ...;Oh-Oh!;Hey!;Ho-Ho!;Whoo-Huh!;3
Fehler in Software werden genannt wie ein...;krabbelndes Insekt;Süßwasserfisch;flugunfähiger
Vogel;Nachtschattengewächse;0
Im Mobilfunk: LTE - Long Term _____;Endorsed;Emitter;Evolution;Example;2
```